

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I **D.VAISHNAVI(192524143)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Design Task

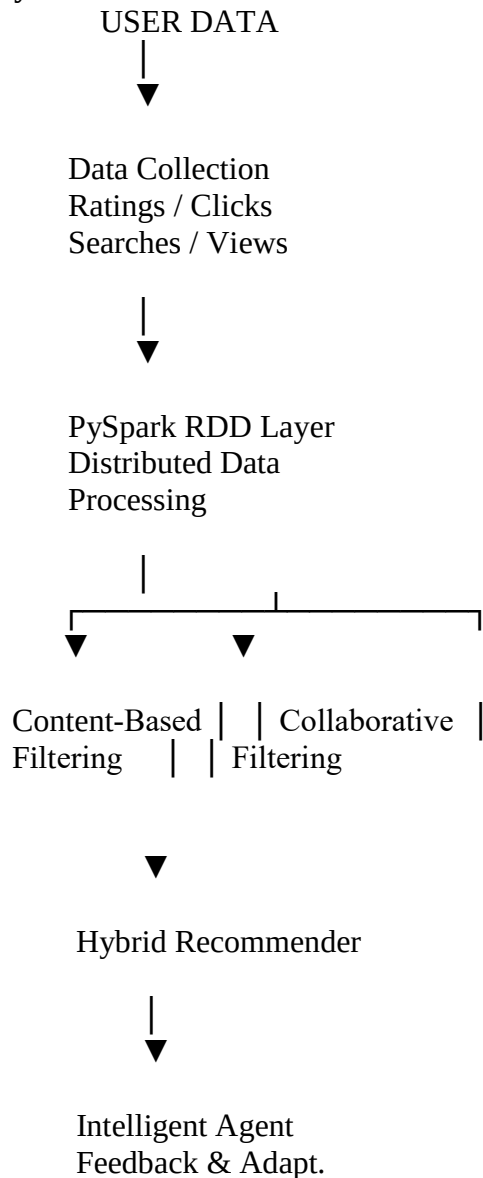
1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

Objective

Design a scalable recommendation system that combines **Content-Based Filtering** and **Collaborative Filtering** using **PySpark RDDs**. The system should analyze large-scale user-item interactions and continuously adapt recommendations according to user behavior and feedback.

System Architecture





Personalized Items

PySpark RDD Processing

RDDs can distribute user-item data across multiple worker nodes.

Example:

```
ratings = sc.textFile("ratings.csv")

data = ratings.map(lambda x: x.split(","))

user_items = data.map(
    lambda x: (x[0], (x[1], float(x[2])))
)

high_rated = user_items.filter(
    lambda x: x[1][1] >= 4
)
```

Common RDD operations include:

- `map()` – transform records
- `filter()` – remove unwanted records
- `flatMap()` – generate multiple records
- `reduceByKey()` – aggregate ratings
- `groupByKey()` – group interactions
- `join()` – combine user/item information
- `sortBy()` – rank recommendations

Recommendation Techniques

Content-Based Filtering

Recommends items similar to those previously liked by the user.

For example:

User liked:

- Action Movies
- Sci-Fi Movies

System recommends:

- Similar Action/Sci-Fi movies

Collaborative Filtering

Uses the behavior of similar users.

User A → Movie 1, Movie 2

User B $\rightarrow$ Movie 1, Movie 2, Movie 3

Therefore:

Movie 3 $\rightarrow$ recommended to User A

Hybrid Approach

The final recommendation score can be calculated as:

$$\begin{aligned} \text{Final Score} = & \\ & \alpha \times \text{Content Score} + \\ & (1-\alpha) \times \text{Collaborative Score} \end{aligned}$$

where α controls the contribution of each technique.

Role of Intelligent Agents

An intelligent recommendation agent:

1. Monitors user behavior.
2. Collects clicks, ratings, searches and purchases.
3. Updates the user's preference profile.
4. Detects changes in interests.
5. Generates personalized recommendations.
6. Learns from positive/negative feedback.

For example:

User watches more AI videos
 $\downarrow$
Agent detects interest
 $\downarrow$
User profile updated
 $\downarrow$
AI-related content gets higher score
 $\downarrow$
New recommendations generated

Benefits

- Handles millions of interactions.
- Distributed processing reduces computation time.
- Combines two recommendation techniques.
- Provides personalized recommendations.
- Continuously learns from feedback.
- Scalable for large applications.

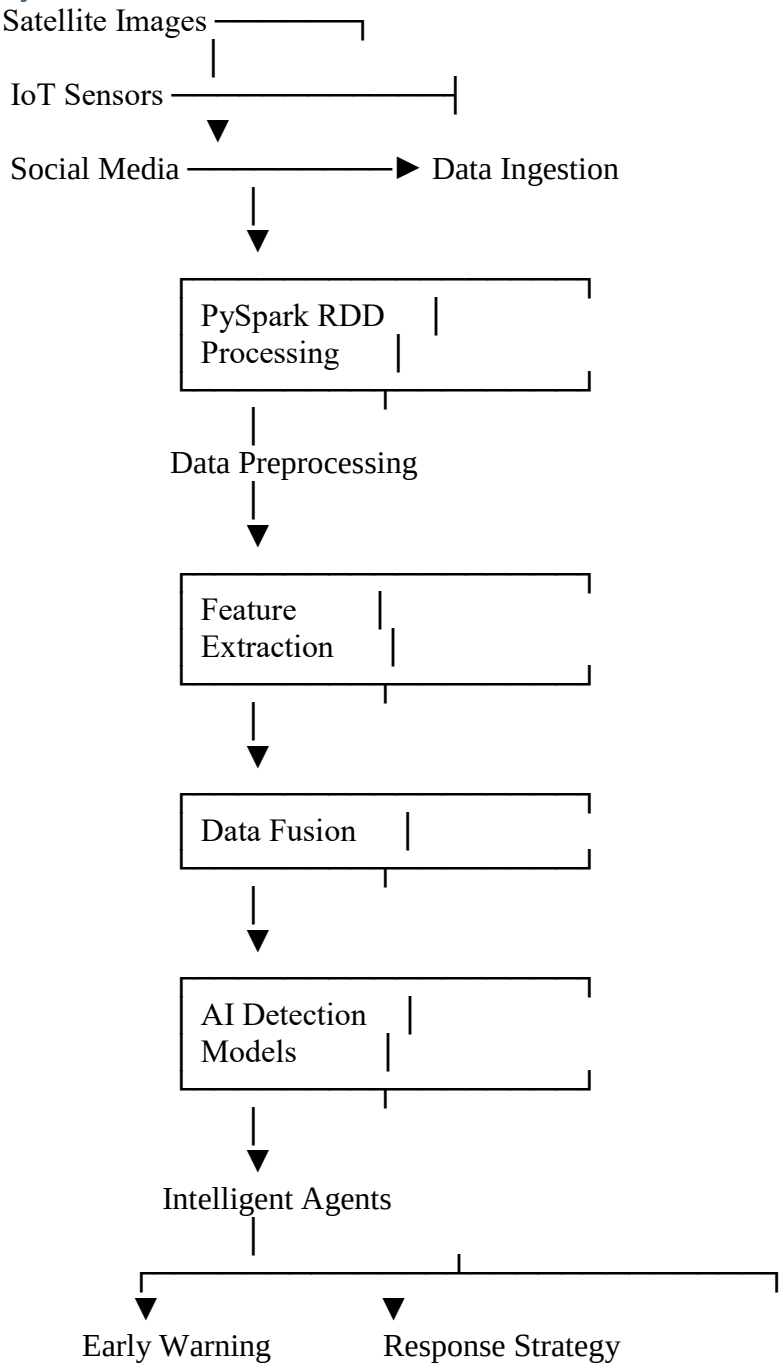
2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

Objective

Develop a distributed AI system using **PySpark RDDs** to detect floods, earthquakes and other natural disasters from **satellite images, IoT sensors and social-media data**.

System Architecture



RDD-Based Processing

```
sensor_data = sc.textFile("sensor_data.csv")
```

```
records = sensor_data.map(  
    lambda x: x.split(",")  
)
```

```
valid_data = records.filter(  
    lambda x: x[2] != "NULL"  
)
```

```
water_levels = valid_data.map(  
    lambda x: (x[0], float(x[2]))  
)
```

```
average_levels = water_levels.reduceByKey(  
    lambda a, b: (a + b) / 2  
)
```

Data Fusion

Data fusion combines information from multiple sources.

Example:

```
Heavy Rainfall  
+  
River Water Level Increasing  
+  
Satellite Shows Water Expansion  
+  
People Report Flooding  
↓  
High Flood Probability
```

Combining multiple sources improves reliability and reduces false alarms.

Intelligent Agents

Different agents can perform specialized tasks:

Sensor Agent

- Monitors sensor readings.
- Detects abnormal measurements.

Satellite Agent

- Analyzes satellite information.
- Identifies affected geographical regions.

Social Media Agent

- Extracts disaster-related posts.
- Identifies location and severity.

Warning Agent

- Calculates disaster risk.
- Sends alerts to authorities and citizens.

Response Agent

- Suggests evacuation routes.
- Identifies emergency resources.

Workflow

Data Collection



RDD Distribution



Data Cleaning



Feature Extraction



Multi-source Data Fusion



AI Disaster Detection



Risk Classification



Agent Coordination



Early Warning



Emergency Response

Benefits

- Early disaster detection.
- Processes large-scale data.
- Combines multiple data sources.
- Reduces false alarms.
- Supports real-time decision making.
- Improves emergency response.

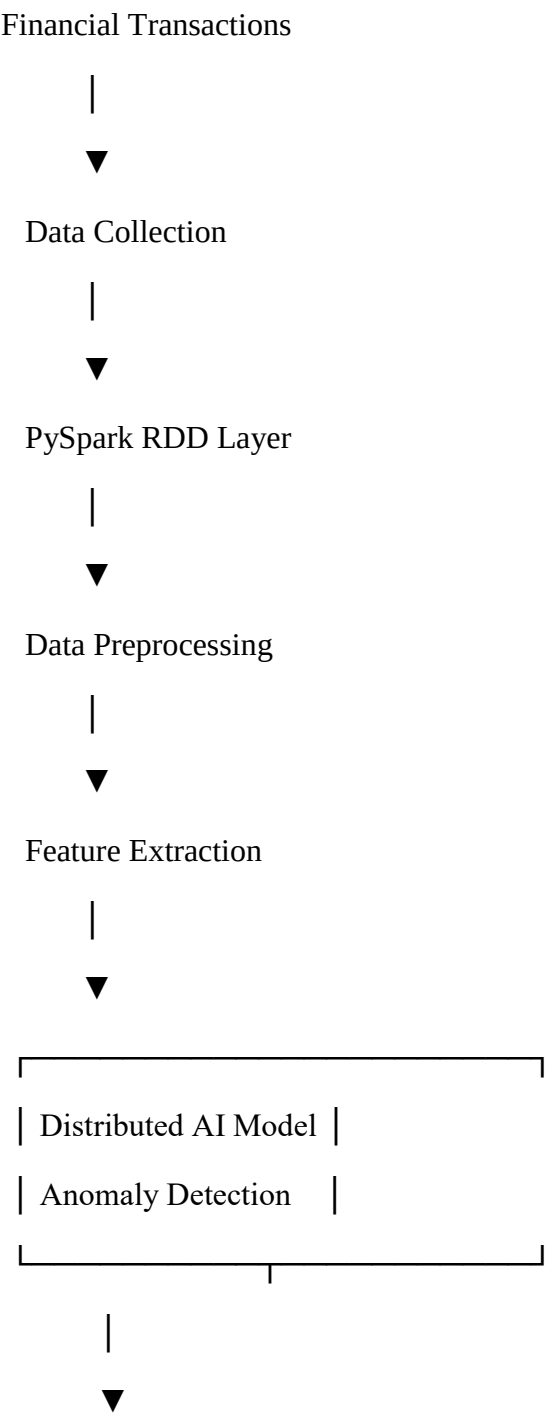
3. AI-Based Fraud Detection Framework

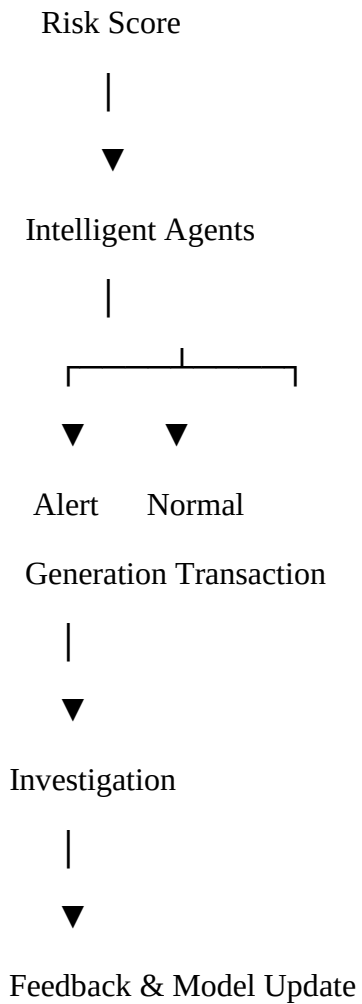
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

Objective

Design a scalable fraud detection framework using **PySpark RDDs** to analyze millions of financial transactions and detect suspicious behavior.

System Architecture





RDD Processing

```
transactions = sc.textFile("transactions.csv")
```

```
data = transactions.map(  
    lambda x: x.split(",")  
)
```

```
large_transactions = data.filter(  
    lambda x: float(x[2]) > 100000  
)
```

```
user_transactions = large_transactions.map(  
    lambda x: (x[1], float(x[2]))  
)
```

```
total_amount = user_transactions.reduceByKey(  
    lambda a, b: a + b  
)
```

Anomaly Detection

The system can calculate features such as:

- Transaction amount.
- Transaction frequency.

- Location change.
- Unusual transaction time.
- Device changes.
- Multiple transactions in a short period.

Example:

Normal behavior:

User → ₹1,000–₹5,000 transactions

Suddenly:

User → ₹2,00,000 transaction

Location → Different country

Time → 3:00 AM

↓

High Anomaly Score

↓

Potential Fraud

[Intelligent Agent Collaboration](#)

Transaction Monitoring Agent

- Monitors incoming transactions.

Anomaly Detection Agent

- Calculates anomaly scores.

Risk Assessment Agent

- Assigns low, medium or high risk.

Alert Agent

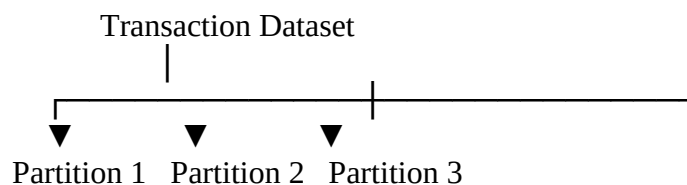
- Generates alerts for suspicious transactions.

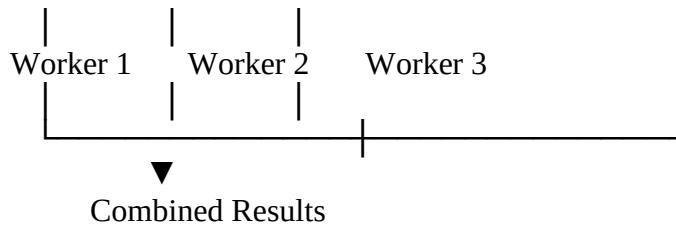
Learning Agent

- Uses confirmed fraud cases to improve future detection.

[Distributed Processing](#)

RDD partitions allow transactions to be processed simultaneously:





Benefits

- Handles massive transaction volumes.
- Fast distributed processing.
- Detects unusual patterns.
- Supports real-time alerts.
- Reduces financial losses.
- Continuously improves through feedback.

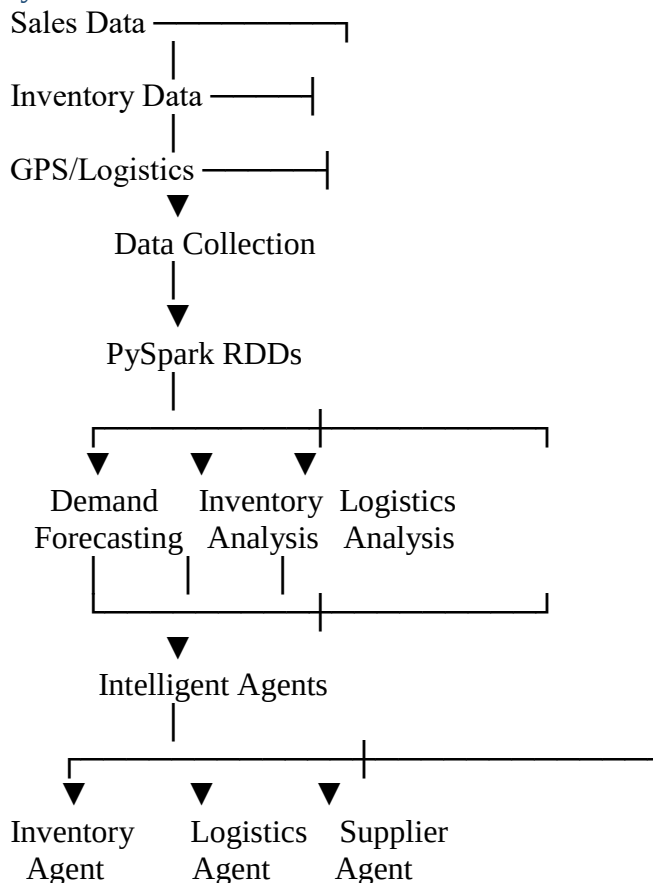
4. Autonomous Supply Chain Optimization System

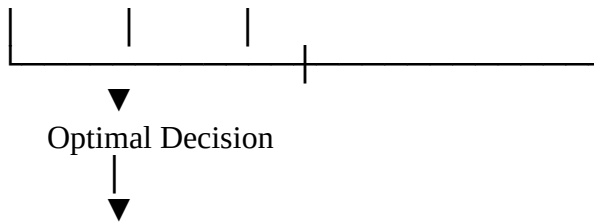
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

Objective

Design a distributed AI system that optimizes **inventory, logistics and demand forecasting** using PySpark RDDs and multiple intelligent agents.

System Architecture





Optimal Decision

Cost & Time Reduction

RDD Transformations

Suppose sales data contains:

Product ID, Quantity, Date, Location

RDD processing:

```
sales = sc.textFile("sales.csv")
```

```
records = sales.map(
    lambda x: x.split(",")
)
```

```
product_sales = records.map(
    lambda x: (x[0], int(x[1]))
)
```

```
total_sales = product_sales.reduceByKey(
    lambda a, b: a + b
)
```

Useful transformations include:

- `map()`
- `filter()`
- `flatMap()`
- `reduceByKey()`
- `groupByKey()`
- `join()`
- `sortByKey()`

Intelligent Agents

Inventory Agent

- Monitors stock.
- Predicts shortages.
- Recommends reorder quantities.

Demand Forecasting Agent

- Analyzes historical sales.
- Predicts future demand.

Logistics Agent

- Selects efficient transportation routes.
- Minimizes delivery time.

Supplier Agent

- Compares suppliers.
- Considers cost and delivery reliability.

Coordination Agent

- Combines decisions from different agents.

Decision Example

Demand Forecast



Expected demand = 10,000 units



Current inventory = 3,000 units



Inventory Agent detects shortage



Supplier Agent selects supplier



Logistics Agent selects optimal route



Order generated

Cost Optimization

The system attempts to minimize:

Total Cost =

Inventory Cost

+ Transportation Cost

+ Storage Cost

+ Ordering Cost

+ Delay Cost

Benefits

- Reduces inventory costs.
- Prevents stock-outs.
- Improves demand forecasting.
- Optimizes transportation.
- Supports decentralized decision-making.
- Handles large supply-chain datasets efficiently.

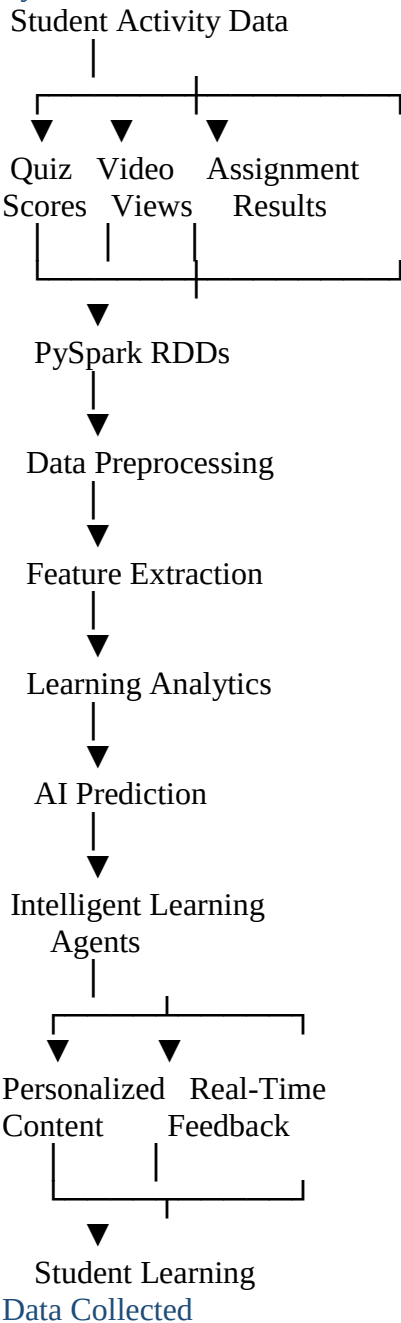
5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

Objective

Develop an AI-based educational platform using **PySpark RDDs** to analyze student learning behavior and performance and provide personalized learning content.

System Architecture



The platform can analyze:

- Quiz scores.
- Assignment marks.
- Video watching time.
- Number of attempts.
- Login frequency.
- Topics completed.
- Questions answered incorrectly.
- Learning speed.

RDD Processing

```
students = sc.textFile("student_data.csv")
```

```
data = students.map(
    lambda x: x.split(",")
)
```

```
scores = data.map(
    lambda x: (x[0], float(x[2]))
)
```

```
average_scores = scores.combineByKey(
    lambda value: (value, 1),
    lambda acc, value: (acc[0] + value, acc[1] + 1),
    lambda acc1, acc2:
        (acc1[0] + acc2[0], acc1[1] + acc2[1])
)
```

RDDs distribute student records among different worker nodes, allowing thousands or millions of learning activities to be processed efficiently.

Intelligent Agents

Student Monitoring Agent

- Tracks student activity.

Performance Agent

- Analyzes marks and learning progress.

Content Recommendation Agent

- Selects appropriate learning materials.

Feedback Agent

- Provides immediate feedback.

Difficulty Adjustment Agent

- Increases or decreases question difficulty based on performance.

Adaptive Learning Example

Student performs poorly in DBMS normalization



Performance Agent detects weakness



Recommendation Agent selects
basic normalization material



Student completes practice questions



Performance improves



Agent increases difficulty

Personalization

Students can be classified according to their learning behavior:

Student Type	Recommended Action
High Performer	Advanced problems
Average Performer	Regular practice
Weak Performer	Basic concepts + tutorials
Inactive Student	Reminders + engagement content
Fast Learner	Higher difficulty

Real-Time Feedback

The platform continuously monitors student activity:

Student Activity



RDD Processing



Performance Analysis



Agent Decision



Personalized Feedback



Updated Student Profile

Benefits

- Supports thousands of students.
- Provides personalized learning.
- Detects weak topics.
- Gives immediate feedback.
- Dynamically adjusts difficulty.
- Improves student engagement.
- Enables data-driven educational decisions.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformation s/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation